PLATEFORME DE RÉFÉRENCE LOCALE

Documentation d’exploitation

DEVOPS LAB LOCAL

Plateforme de référence miniature - projet de stage chez Sobrus

Identification	Valeur
Code	DEVOPS-LAB-EXPLOITATION-1.0
Version / date	1.0 - 19 octobre 2026
Périmètre	Windows, Docker Desktop / WSL 2, Kubernetes kind
Dépôt	https://github.com/yazid9871/Devops-lab
Statut	Document de travail, à relire avec l’encadrement
Approbation	Aucune approbation formelle enregistrée

> État de livraison : le socle dispose de validations précédentes. Les ajouts Helm, Traefik et cert-manager sont préparés dans les fichiers, mais leur déploiement et leurs tests fonctionnels restent à réaliser.

Une seule machine. Des données fictives. Aucun accès à la production. Les procédures décrivent le lab et ne constituent pas des instructions de maintenance de l’infrastructure Sobrus.

Suivi documentaire et sommaire

Version	Évolution
1.0	Documentation du socle et des évolutions Helm / HTTPS préparées.

Ce document reprend les thèmes de la documentation d'exploitation Sobrus pour les adapter au projet local. Le cahier des charges principal reste « Plateforme de référence locale (2 jours) ». La présentation retenue dure 10 minutes, selon la consigne actualisée.

Comment lire les statuts

Statut	Interprétation
Historique documenté	Un compte rendu ou un rapport existe ; cela ne prouve pas l'état actuel du cluster.
Préparé	Les fichiers existent ; le déploiement ou le scénario n'a pas été validé.
Attendu	Résultat à constater lors de l'exécution ultérieure de la procédure.

Sommaire

1. Objectif et périmètre	3
2. Schéma d'architecture	4
3. Organisation Kubernetes	5
4. Préparer Windows	6
5. Installation et accès	7
6. Registry et publication d'images	8
7. GitOps, ArgoCD et Helm	9
8. Runbook : déployer et revenir en arrière	10
9. Traefik, certificats et accès HTTPS	11
10. Exploitation de PostgreSQL et Redis	12
11. Runbook : sauvegarder et restaurer	13
12. Stockage et persistance	14
13. Supervision et règles d'alerte	15
14. Runbook : diagnostiquer un incident	16
15. Sécurité et accès opérateur	17
16. Validation, preuves et réception	18
17. Limites, références et reprise	19
Annexe. Accès aux interfaces	20

1. Objectif et périmètre

Le lab reproduit en miniature une chaîne de déploiement et des opérations d'exploitation : publication d'image, synchronisation GitOps, retour arrière, sauvegarde, restauration et supervision. Le README et le runbook doivent permettre à un collègue de reprendre le travail.

Livrables et critères

Attendu	Réponse du projet
Déploiement et rollback	Application Python, registry local, GitHub et ArgoCD.
Données	PostgreSQL et Redis d'exercice, avec identifiants générés localement.
Sauvegarde et restauration	CronJob pg_dump, archive contrôlée, restauration sur une nouvelle cible.
Supervision	Prometheus, Grafana, Alertmanager et règles d'alerte.
Reproductibilité	Installation PowerShell documentée ; réception sur un autre PC à faire.
Documentation	README, runbook, schéma, comptes rendus et note de recul.

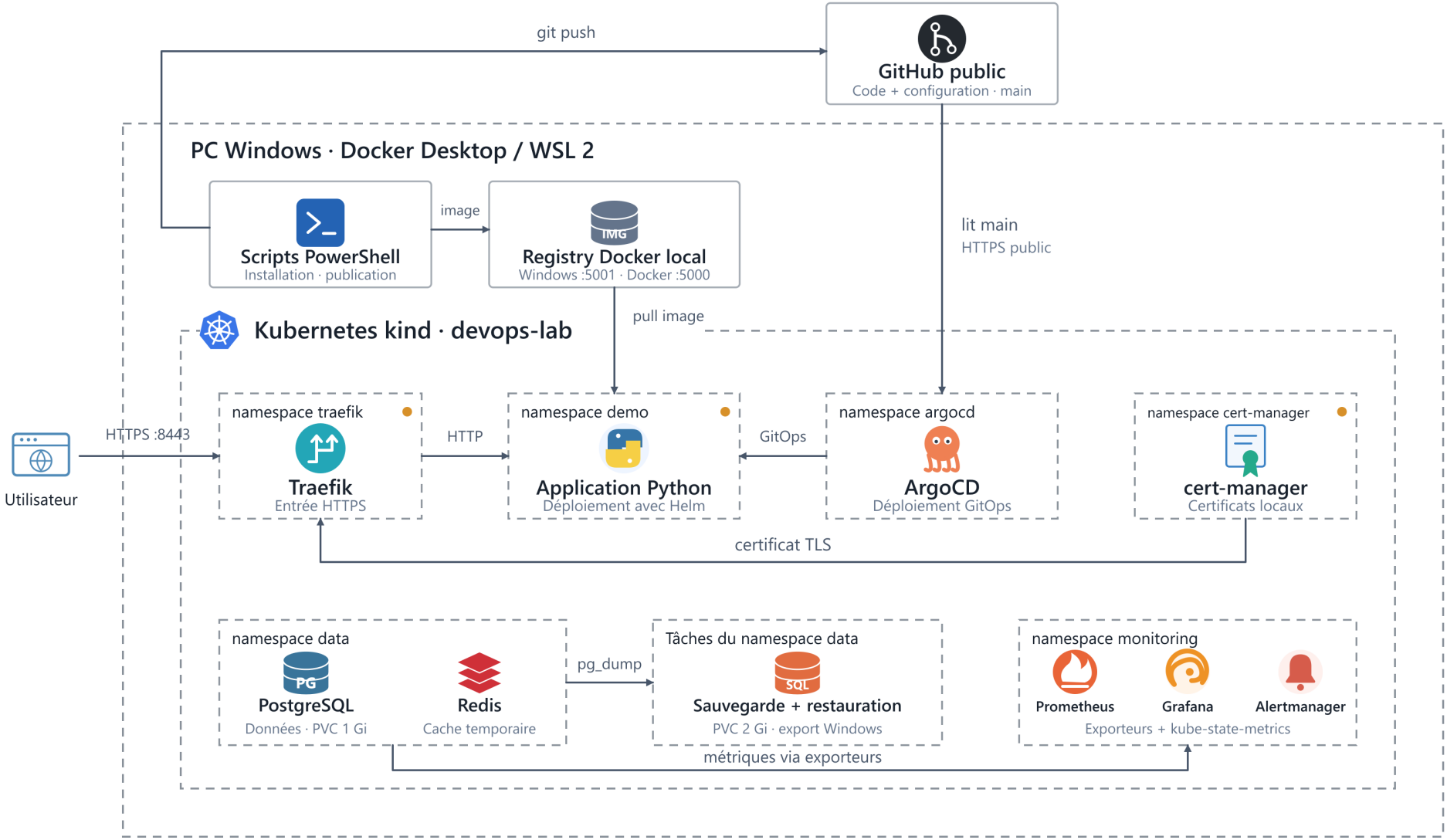
Adaptation des technologies Sobrus

Kubernetes, ArgoCD, Prometheus et Grafana constituent le socle commun. Helm, Traefik et cert-manager rapprochent la configuration de la référence, mais sont encore préparés. PostgreSQL et pg_dump sont explicitement permis par le cahier des charges ; MySQL et Velero ne sont pas obligatoires.

> Le fonctionnement des services est local. GitHub, les charts officiels et les registres d'images restent externes : le parcours n'est pas autonome hors ligne.

Ceph/Rook, Restic/S3, Velero, Harbor, Trivy, Falco et le VPN ne sont pas intégrés à ce périmètre. Leur absence doit être expliquée, sans les présenter comme des réalisations du lab.

2. Schéma d'architecture



Points orange : ajouts Helm / HTTPS préparés, non déployés. Application web indépendante de PostgreSQL et Redis.

3. Organisation Kubernetes

Namespace	Composants et gestion
argocd	Contrôleurs et serveur ArgoCD ; quatre Applications sur main, projet default.
demo	Application Python ; chart Helm préparé avec Deployment, Service, Ingress et Certificate.
data	PostgreSQL, Redis, exporteurs, CronJob et cibles de restauration.
monitoring	Prometheus, Grafana, Alertmanager, kube-state-metrics et ConfigMaps.
traefik	Contrôleur et Service ClusterIP ; release Helm préparée.
cert-manager	Contrôleur, webhook, cainjector et autorité locale ; préparation non déployée.

Objets à connaître

Objet	Explication
Pod	Unité d'exécution qui contient un ou plusieurs conteneurs.
Deployment	Maintient le nombre de pods attendu pour un service, comme l'application.
StatefulSet	Gère une identité stable et le stockage d'un service avec état, comme PostgreSQL.
Service	Fournit une adresse et un port stables pour joindre les pods.
PVC	Demande de stockage persistant, indépendante du remplacement d'un pod.
CronJob / Job	Planification puis exécution d'une tâche ponctuelle.
Ingress / Certificate	Règle d'entrée web et demande de certificat TLS.

Contexte : kind-devops-lab. Cluster : devops-lab. Le script de création référence kindest/node:v1.37.0. Le DNS des services est assuré par CoreDNS et le stockage standard par local-path.

> Un seul nœud : la séparation en namespaces n'apporte ni haute disponibilité entre machines ni isolation réseau automatique. Le projet ne définit pas de NetworkPolicy.

4. Préparer Windows

Parcours prévu : Windows 11 Intel/AMD 64 bits, Internet et droits administrateur pour préparer WSL. Dans le Gestionnaire des tâches, vérifier « Performances > Processeur > Virtualisation : Activée ». Sinon, activer VT-x ou SVM dans le BIOS, enregistrer puis redémarrer.

WSL 2 - PowerShell administrateur

```
wsl --install --no-distribution
```

Redémarrer Windows, puis rouvrir PowerShell administrateur :

```
wsl --update
wsl --set-default-version 2
wsl --version
```

Installer les outils avec WinGet

Exécuter une commande à la fois. Si winget est introuvable, installer ou mettre à jour App Installer depuis le Microsoft Store, puis rouvrir le terminal.

```
winget --version
winget install --exact --id Git.Git --source winget
winget install --exact --id Docker.DockerDesktop --source winget
winget install --exact --id Kubernetes.kind --source winget
winget install --exact --id Kubernetes.kubectl --source winget
winget install --exact --id Helm.Helm --source winget
```

Avant de poursuivre

Fermer et rouvrir PowerShell après installation. Démarrer Docker Desktop, choisir le moteur WSL 2 et les conteneurs Linux, puis attendre son démarrage. Ne pas utiliser son Kubernetes intégré : kind crée le cluster du lab.

```
docker version
docker --context desktop-linux info --format '{{.OSType}}'
git --version
kind version
kubectl version --client
helm version --short
```

Attendu : les versions s'affichent, Docker présente Client et Server, et son type de système est linux. La préparation Windows est mesurée séparément de l'objectif des 30 minutes.

5. Installation et accès

> La configuration Helm / HTTPS doit d'abord être publiée sur GitHub avant de lancer cette version de l'installation. À la rédaction, ces ajouts restent locaux et non validés.

Cloner et lancer - depuis PowerShell

```
Set-Location ([Environment]::GetFolderPath('Desktop'))
git clone https://github.com/yazid9871/Devops-lab.git
Set-Location .\Devops-lab
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\scripts\Install-Lab.ps1
```

La lecture publique en HTTPS ne demande ni clé SSH ni compte GitHub. Le script vérifie les outils, origin, main, la propreté du dépôt et sa synchronisation. Il prépare ensuite le cluster, les accès, l'application, les données, la restauration et la supervision.

Résultat et rapport

Attendu : PASS et rapport work/install-report.json. Ce rapport mesure la commande ; pour la réception, chronométrer du clonage jusqu'à l'ouverture de Grafana, outils déjà prêts. Aucune durée sur un PC neuf n'est encore validée.

Chaque relance du scénario de restauration ajoute une ligne fictive et conserve une nouvelle instance avec son volume. Une relance ne doit pas être utilisée comme méthode de nettoyage.

Ouvrir les interfaces - un terminal par tunnel

```
.\scripts\Open-Demo.ps1
# Application : https://localhost:8443

kubectl --context kind-devops-lab -n monitoring `
  port-forward svc/grafana 3000:3000
# Grafana : http://localhost:3000/d/devops-lab
```

Garder le terminal occupé ouvert. Ctrl+C ferme uniquement le tunnel. Les commandes complètes d'accès figurent dans docs/acces-interfaces.md. Ne pas afficher les mots de passe pendant une démonstration ou dans les comptes rendus.

6. Registry et publication d'images

Le registry local stocke les images applicatives. Il s'agit d'un conteneur Docker nommé devops-lab-registry, lié au réseau kind, avec un volume nommé devops-lab-registry-data. Ce n'est ni Docker Hub ni une instance Harbor.

Point de vue	Adresse et fonction
Windows	localhost:5001 : publication de l'image et API du registry.
Nœud kind	devops-lab-registry:5000 : accès au même conteneur sur le réseau Docker.
Image du Deployment	localhost:5001/devops-demo:vN : référence versionnée de l'application.

Correspondance des adresses

infra/registry/hosts.toml indique à containerd comment résoudre et télécharger l'image depuis le registry du réseau Docker. Le script Setup-Registry.ps1 prépare cette configuration, conserve une copie de la configuration containerd avant modification et ne recrée pas le cluster.

Contrôles de diagnostic à exécuter au besoin

```
docker --context desktop-linux ps --filter name=devops-lab-registry
Invoke-RestMethod http://127.0.0.1:5001/v2/devops-demo/tags/list
```

Attendu : conteneur en fonctionnement et liste de tags. Une erreur de téléchargement Kubernetes doit être diagnostiquée aussi côté containerd ; un push réussi depuis Windows ne prouve pas à lui seul l'accès du nœud.

Limites

Le port Windows écoute sur 127.0.0.1, en HTTP, sans authentification. Les images restent sur le PC. Les tags sont préservés par le script de publication, mais l'image applicative n'est pas référencée par digest dans le chart.

> Deux PC suivant le même dépôt ont deux registries distincts. Un changement Git peut parvenir aux deux clusters, alors que la nouvelle image n'existe que sur le PC qui l'a publiée.

7. GitOps, ArgoCD et Helm

ArgoCD lit la configuration dans GitHub et la compare aux objets du cluster. Il synchronise les changements et corrige les écarts avec selfHeal. Il ne construit pas les images et ne se trouve pas dans le chemin du trafic utilisateur.

Application	Source / destination
devops-demo	charts/devops-demo : rendu Helm, releaseName devops-demo, namespace demo.
postgres-lab	k8s/postgres : Kustomize, namespace data.
redis-lab	k8s/redis : Kustomize, namespace data.
monitoring-lab	k8s/monitoring : namespace monitoring et exporteur PostgreSQL dans data.

Toutes suivent main et le projet default. prune: false désactive la suppression automatique des objets retirés de Git ; des objets anciens peuvent donc rester dans le cluster. Les scripts remplacent OWNER/REPOSITORY en mémoire à partir de origin.

Structure du chart applicatif

Chart.yaml identifie le chart (version 0.2.0). values.yaml contient l'image, les répliques, les ressources et les paramètres HTTPS. templates/ génère Namespace, Deployment, Service, Ingress et Certificate. Le schéma JSON contraint les valeurs du lab.

app/VERSION et image.tag doivent correspondre. ArgoCD effectue le rendu Helm puis gère les ressources. L'application n'est pas une release administrée par helm upgrade ou helm rollback. Traefik et cert-manager ont, eux, des releases Helm explicites installées par script.

Transition préparée - à exécuter après publication

```
.\scripts\Deploy-Access.ps1
.\scripts\Deploy-Demo.ps1
```

> Préparé, non déployé : l'ancien chemin k8s/demo est remplacé par charts/devops-demo dans les fichiers locaux. Les noms devops-demo et demo sont conservés. Aucun test fonctionnel de cette migration n'est déclaré réussi.

8. Runbook : déployer et revenir en arrière

Prérequis et effets

Le lab doit être installé, la migration préparée publiée et déployée, main synchronisée avec origin/main, le dépôt propre et l'identité Git configurée. Le compte doit pouvoir pousser dans le dépôt. La publication crée une image et un commit ; elle déclenche le déploiement ArgoCD.

```
git status --short
git branch --show-current
.\scripts\Publish-Demo.ps1
if (-not $?) { throw 'Publication en echec : arreter ici.' }
$deployCommit = git rev-parse HEAD
.\scripts\Test-Deployment.ps1
```

Attendu après validation future : image publiée, ArgoCD Synced / Healthy, version HTTP conforme et santé ok. La chaîne actuelle prépare aussi les contrôles du chart Helm ; elle est déclenchée manuellement, sans workflow CI hébergé.

Retour à la version précédente - même terminal

```
git revert --no-edit $deployCommit
if ($LASTEXITCODE -ne 0) { throw 'Revert en echec : arreter ici.' }
git push origin main
if ($LASTEXITCODE -ne 0) { throw 'Push en echec : arreter ici.' }
.\scripts\Test-Deployment.ps1
```

Attendu : ancien tag et ancienne version. Si le terminal a été fermé, retrouver le commit de publication dans git log --oneline et utiliser son SHA. Ne pas sélectionner le commit de migration Helm ni un commit déjà annulé.

Si l'opération échoue

Conserver la sortie, examiner git status et git diff, puis les conditions de l'Application ArgoCD. Un échec avant commit peut laisser VERSION et values.yaml modifiés. Ne pas effacer ces changements sans examen. Le rollback nécessite l'ancienne image et ne traite pas une migration de schéma PostgreSQL.

9. Traefik, certificats et accès HTTPS

> Procédures préparées, non exécutées : émission, confiance navigateur, routage HTTPS et renouvellement restent à valider.

Traefik utilise un Service ClusterIP et l'IngressClass traefik-lab. Le chart préparé demande le host localhost sur l'entrée websecure. Le tunnel relie 127.0.0.1:8443 au Service Traefik:443 ; Traefik termine TLS puis contacte le Service applicatif en HTTP.

Élément	Configuration préparée
Traefik	Chart 41.6.1 ; valeurs dans infra/access/traefik-values.yaml.
cert-manager	Chart v1.21.2 ; CRD conservées ; valeurs dans infra/access/.
Autorité	Certificate racine dans cert-manager et ClusterIssuer devops-lab-ca.
Certificat serveur	demo/devops-demo-tls ; localhost ; 90 jours, renouvellement 15 jours avant expiration.

Ouvrir et approuver explicitement l'autorité

```
.\scripts\Open-Demo.ps1
# Dans un autre terminal, si la confiance navigateur est souhaitee :
.\scripts\Export-LabCA.ps1 -Trust
# Fermer puis rouvrir le navigateur : https://localhost:8443
```

Le certificat public est exporté dans work/tls/. L'option -Trust l'importe uniquement dans Cert:\CurrentUser\Root. L'installation ne modifie pas cette confiance automatiquement et n'exporte aucune clé privée.

Validation ultérieure - tunnel ouvert

```
.\scripts\Test-Https.ps1
kubectl --context kind-devops-lab -n demo `
  describe certificate devops-demo-tls
```

Attendu : vérification du nom et de l'autorité, santé ok et bonne version. Le test ne désactive pas la vérification TLS. Le rapport d'installation laisse HttpsEndpointTested=false : un certificat prêt ne prouve pas une requête HTTPS réussie.

Une autorité locale remplace l'émetteur public pour cet exercice. Le renouvellement configuré n'a pas encore été observé. La distribution et le remplacement de l'autorité restent manuels ; aucun service de révocation n'est intégré.

10. Exploitation de PostgreSQL et Redis

Service	Organisation
PostgreSQL	StatefulSet postgres, une réplique postgres-0 ; base et utilisateur lab ; table fictive demo_items.
Accès PostgreSQL	Service headless, postgres.data.svc.cluster.local:5432 ; authentification du Secret postgres-credentials.
Persistance PostgreSQL	PVC de 1 Gi, classe standard ; init par ConfigMap sur instance neuve.
Redis	Deployment à une réplique ; Redis et redis-exporter dans le même pod.
Accès Redis	Service redis:6379 ; mot de passe du Secret redis-credentials ; métriques sur 9121.
Cache Redis	RDB/AOF désactivés, emptyDir, limite Redis 96 Mio et politique allkeys-lru.

Contrôles disponibles pour l'utilisateur

```
.\scripts\Test-Postgres.ps1
.\scripts\Test-Redis.ps1
kubectl --context kind-devops-lab -n data get pods,pvc
```

Attendu pour PostgreSQL : authentification TCP, écriture/lecture dans une table temporaire annulée, empreinte des données et volume Bound. Attendu pour Redis : refus sans authentification, PING, écriture/lecture et expiration d'une clé de contrôle unique.

Précautions d'exploitation

Ne pas régénérer le mot de passe d'une base existante pour résoudre un problème de connexion. Si son Secret manque alors qu'un volume existe, récupérer le Secret d'origine. Lire les événements et les logs avant une correction.

```
kubectl --context kind-devops-lab -n data logs postgres-0 --tail=50
kubectl --context kind-devops-lab -n data `
  logs deployment/redis -c redis --tail=50
```

> L'application web n'utilise pas encore ces services. Redis est un cache temporaire sans sauvegarde. Les commandes de contrôle ne doivent pas être présentées comme une validation d'intégration applicative.

11. Runbook : sauvegarder et restaurer

Le CronJob postgres-backup lance pg_dump -Fc toutes les cinq minutes, en UTC, avec concurrencyPolicy: Forbid. Les archives sont stockées sur un PVC distinct de 2 Gi. Le script écrit une archive temporaire, vérifie sa structure, puis produit le SHA-256 et un marqueur de fin.

Créer et exporter une sauvegarde

```
.\scripts\Backup-Postgres.ps1
```

Attendu : Job réussi, archive copiée dans backups/ sur Windows et SHA-256 conforme. L'export vers Windows est manuel ; seule la sauvegarde dans le volume est planifiée.

Scénario de restauration contrôlée

```
.\scripts\Test-Restore.ps1
```

Effets : ajout d'une ligne fictive à la source, nouvelle sauvegarde, nouvelle instance PostgreSQL et nouveau PVC de 1 Gi. Le script refuse une cible non vide, vérifie le SHA-256, restaure puis compare les données et le schéma. La source n'est pas remplacée.

À conserver	Preuve attendue
Archive	Répertoire snapshot, contrôle SHA-256 et marqueur completed.
Cible	Nom unique postgres-restore-... ; nouvelle instance et volume distinct.
Intégrité	Nombre de lignes et empreintes identiques ; colonnes, contraintes et index comparés.
Durée	Rapport backups/*-report.json ; temps pg_restore et durée du scénario après sauvegarde.

Échec et limites

Lire les logs du Job de sauvegarde ou restauration et l'état du PVC. Conserver la cible en échec pour diagnostic. Chaque relance conserve une nouvelle instance ; aucune suppression automatique n'est prévue.

> Ce scénario ne sélectionne pas une ancienne archive, ne bascule pas l'application et ne reconstruit pas les rôles globaux, propriétaires ou ACL. Les sauvegardes restent sur le même PC. Une restauration SQL n'est pas une restauration complète du cluster.

12. Stockage et persistance

Contenu	Support	Limite
PostgreSQL source	PVC 1 Gi	Même nœud et même PC.
Archives SQL	PVC 2 Gi	Rétention des archives non automatisée.
Cible de restauration	PVC 1 Gi par cible	Les cibles sont conservées.
Prometheus	PVC 2 Gi	Métriques limitées à 6 h et 1 Go.
Grafana	PVC 1 Gi	Base locale ; dashboard aussi décrit dans Git.
Images Docker	Volume Docker nommé	Hors Kubernetes, sur le même PC.
Redis / Alertmanager	emptyDir	État éphémère au remplacement du pod.
Certificats / autorité	Secrets dans etcd	Non inclus dans les dumps PostgreSQL.
Exports Windows	backups/ et work/	Hors Git, pas de copie hors site.

Le socle demande 6 Gi de PVC, hors cibles de restauration, images et stockage système. Il s'agit de capacités demandées, pas d'une mesure d'occupation. Chaque cible supplémentaire demande 1 Gi.

Commandes de lecture

```
kubectl --context kind-devops-lab get storageclass
kubectl --context kind-devops-lab -n data get pvc
kubectl --context kind-devops-lab -n monitoring get pvc
docker --context desktop-linux volume ls
```

Attendu pour les volumes utilisés : Bound. Le provisionneur local-path stocke les données dans le nœud kind. La persistance après remplacement d'un pod ne démontre pas une reprise après perte du nœud ou du PC.

> Ne supprimer ni cluster, ni volume, ni donnée sans accord. Git et prune: false ne protègent pas d'une perte physique des données. Prévoir une copie externe avant d'envisager un usage dépassant l'exercice local.

13. Supervision et règles d'alerte

Prometheus collecte les métriques toutes les 15 secondes et évalue les règles toutes les 15 secondes. Grafana interroge Prometheus pour afficher les tableaux de bord. Alertmanager reçoit et regroupe les alertes ; aucun envoi externe n'est configuré.

Cible	Métriques exposées
Prometheus :9090	État de Prometheus lui-même.
kube-state-metrics :8080	État des objets Kubernetes, pods, Jobs et CronJobs.
postgres-exporter :9187	État de PostgreSQL ; exporteur dans data.
redis-exporter :9121	État de Redis ; exporteur dans le pod Redis.

Alerte	Interprétation
LabPodError	Pod Failed ou attente en erreur dans demo/data depuis 1 minute.
LabPostgresUnavailable	Base, exporteur ou métrique indisponible depuis 1 minute.
LabBackupFailed	Job échoué, aucun succès depuis 15 min ou métrique absente ; délai de confirmation 1 min.
LabRedisUnavailable	Redis/exporteur indisponible alors que Redis est demandé actif ; confirmation 1 min.

Consulter les alertes

```
kubectl --context kind-devops-lab -n monitoring `
  port-forward svc/prometheus 9090:9090
# Ouvrir http://localhost:9090/alerts
```

pending signifie que la condition attend confirmation ; firing signifie que le délai configuré est écoulé. La durée réelle inclut aussi la collecte et l'évaluation.

> Le tableau Grafana inventorie tous les namespaces. La règle LabPodError couvre seulement demo et data. Les métriques propres de Traefik et cert-manager ne sont pas ajoutées aux quatre cibles actuelles. Les tests simulés ne prouvent pas une panne réelle de bout en bout.

14. Runbook : diagnostiquer un incident

Première lecture - sans modifier le lab

```
kubectl --context kind-devops-lab get nodes
kubectl --context kind-devops-lab get pods -A
kubectl --context kind-devops-lab -n argocd get applications
kubectl --context kind-devops-lab -n data get cronjob,jobs,pvc
```

Un pod de sauvegarde Completed est normal. Après redémarrage de Docker, certains composants peuvent reprendre progressivement ; relever les événements avant de conclure à une panne persistante.

Symptôme	Piste et contrôle
ImagePullBackOff	Registry, tag demandé et résolution containerd ; lire describe pod.
CrashLoopBackOff	Logs du conteneur et événements ; identifier la cause avant de remplacer le pod.
ArgoCD non synchronisé	Accès public HTTPS, branche, commit et conditions de l'Application.
Base indisponible	Pod postgres-0, exporteur, Secret, PVC et logs.
Sauvegarde en erreur	Logs du Job, CronJob actif et espace disponible ; vérifier le dernier succès.
HTTPS refusé	Tunnel Open-Demo, port local, Deployment et Service Traefik.
HTTPS 404 / 503	404 : host localhost et classe Ingress ; 503 : Service, pods et endpoints.
Certificat non reconnu	Ready du Certificate, autorité du PC et confiance explicite de l'utilisateur.

Exemples de collecte de diagnostic

```
kubectl --context kind-devops-lab -n demo `
  logs deployment/devops-demo --tail=50
kubectl --context kind-devops-lab -n demo `
  describe certificate devops-demo-tls
kubectl --context kind-devops-lab -n traefik `
  logs deployment/traefik --tail=50
```

Après correction, relancer uniquement le contrôle concerné. Ne pas supprimer un volume, un Secret d'autorité ou une CRD pour contourner le problème. Un ancien Job échoué peut garder l'alerte de sauvegarde active tant qu'il est conservé.

15. Sécurité et accès opérateur

Gestion des secrets

Les Secrets postgres-credentials, redis-credentials et lab-grafana-admin sont générés localement. L'autorité et le certificat serveur restent aussi dans des Secrets Kubernetes. Leur contenu ne doit pas être committé, exporté dans les comptes rendus ou affiché à l'écran pendant la présentation.

Secrets encodés en base64 ne signifie pas coffre-fort chiffré. Le projet ne configure pas le chiffrement au repos d'etcd. L'opérateur conserve des droits d'administration larges sur le lab.

Protection des conteneurs

L'application est configurée sans root, avec système de fichiers en lecture seule, capabilities supprimées et token ServiceAccount désactivé. Les workloads de données appliquent aussi plusieurs restrictions. Les contrôleurs ont les accès API nécessaires à leur fonction.

Lecture et écriture du dépôt

Le clonage et la lecture ArgoCD du dépôt public ne demandent pas d'identifiant. La publication et le rollback Git nécessitent un compte autorisé à écrire. Les anciens fichiers ou Secrets SSH ne sont pas supprimés par le nouveau parcours ; leur éventuelle présence ne valide pas leur usage actuel.

Confiance TLS et limites réseau

Le certificat public de l'autorité peut être approuvé pour l'utilisateur Windows courant avec Export-LabCA.ps1 -Trust. Noter son empreinte. Pour retirer cette confiance, utiliser certmgr.msc, retrouver uniquement la racine du lab, vérifier l'empreinte puis retirer cette entrée.

HTTPS se termine dans Traefik. Le trafic vers l'application, les accès de données et le registry restent locaux sans généralisation de TLS. Les namespaces ne remplacent pas des NetworkPolicies.

> Le lab n'est pas une configuration de production. La révocation des certificats, le renouvellement de confiance des postes, la restauration de l'autorité et une séparation fine des droits restent à définir avant un pilote réel.

16. Validation, preuves et réception

Élément	Preuve disponible / état
Déploiement / rollback	Compte rendu du 16 octobre : publication et retour arrière ArgoCD avec l'ancien parcours Kustomize.
Installation	16 octobre : 1,53 min sur lab existant, images en cache et clé SSH déjà autorisée.
Supervision	16 octobre : collecte et règles simulées documentées ; aucune notification externe ni panne réelle de bout en bout validée.
Restauration récente	Rapport du 19 octobre : 7 lignes comparées, cible vide, 40 ms de pg_restore et 12,71 s après sauvegarde.
HTTPS public / Helm / TLS	Modifications préparées ; syntaxe PowerShell/JSON vérifiée, sans rendu Helm ni test fonctionnel des ajouts.
Second PC	Relance avec README seul et durée totale : à réaliser.

Les durées mesurées sur quelques lignes fictives et un cluster existant ne prédisent pas une restauration de production ou une installation à froid. Aucun nouveau test du lab n'a été lancé pour rédiger cette documentation.

Réception à faire par l'utilisateur, après finalisation

Préparer Windows et les outils. Publier la version choisie. Sur l'autre PC, démarrer le chronomètre avant le clonage, suivre uniquement le README, puis arrêter après PASS et l'ouverture de Grafana. Relever OS, ressources Docker, connexion, commit, durée et blocages. Noter toute explication orale indispensable.

Valider séparément la publication/rollback, le HTTPS et la restauration ; conserver leur sortie et leur contexte. Les ressources des contrôleurs et leur compatibilité effective restent à confirmer sur le cluster utilisé.

> Le rapport du 19 octobre est dans backups/postgres-restore-20261019005425-98a181-report.json. Le temps global indiqué exclut la sauvegarde. Le fichier reste hors Git.

17. Limites, références et reprise

Ce qui est transposable

Versionner la configuration, déployer avec ArgoCD, conserver un historique Git, contrôler une sauvegarde par restauration, centraliser les métriques et documenter le diagnostic constituent les méthodes démontrées ou préparées par le lab.

Ce qui reste hors du périmètre validé

Haute disponibilité, perte complète du PC, sauvegarde hors site, rétention d'archives automatisée, reprise d'une ancienne archive choisie, bascule applicative, migrations de schéma, notifications externes et charge représentative ne sont pas validées. L'application ne consomme pas encore les services de données.

Documents du dépôt à utiliser

Document	Usage
README.md	Installation et commandes des prérequis.
docs/runbook.md	Déploiement, rollback, restauration, alertes et TLS.
docs/architecture.md	Schémas détaillés, ports, responsabilités et stockage.
docs/helm.md / https-local.md	Migration préparée et accès HTTPS local.
docs/validation-*.md	Comptes rendus datés et limites des validations.
docs/note-de-recul.md	Applicabilité à l'infrastructure réelle et risques.
docs/presentation-10-minutes.md	Déroulé de démonstration.

Références utilisées

Cahier des charges fourni : « Plateforme de référence locale (2 jours) », 2 pages. Référence de structure : « Documentation exploitation OVH Kubernetes », Sobrus / LecPac, version 1.2, 28 pages. Leurs consignes et adresses d'exploitation ne sont pas des commandes à exécuter sur le lab.

Sources techniques du projet : README, scripts et manifests locaux, schéma simplifié et rapport de restauration du 19 octobre. Base Git locale publiée connue : 794d4b2 ; les évolutions Helm/HTTPS sont encore non committées à la rédaction.

Documentation officielle : helm.sh ; kind.sigs.k8s.io ; kubernetes.io ; argo-cd.readthedocs.io ; cert-manager.io ; github.com/traefik/traefik-helm-chart. Les liens détaillés figurent dans les guides du dépôt.

Annexe. Accès aux interfaces

Lancer les commandes depuis le dossier du projet. Chaque port-forward occupe son terminal : utiliser un terminal séparé par interface et le garder ouvert. Ctrl+C ferme seulement ce tunnel.

Application

```
.\scripts\Open-Demo.ps1
# https://localhost:8443 - apres deploiement de l'evolution HTTPS
```

Pour l'accès HTTP direct de diagnostic :

```
kubectl --context kind-devops-lab -n demo `
  port-forward svc/devops-demo 8081:80
# http://localhost:8081
```

Grafana - compte admin

```
kubectl --context kind-devops-lab -n monitoring `
  port-forward svc/grafana 3000:3000
# http://localhost:3000/d/devops-lab
```

Dans un autre terminal, afficher le mot de passe local uniquement si nécessaire, hors partage d'écran :

```
$encoded = kubectl --context kind-devops-lab -n monitoring `
  get secret lab-grafana-admin -o jsonpath='{.data.admin-password}'
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($encoded))
```

ArgoCD - compte admin

```
kubectl --context kind-devops-lab -n argocd `
  port-forward svc/argocd-server 8080:443
# https://localhost:8080 - certificat local distinct
$encoded = kubectl --context kind-devops-lab -n argocd `
  get secret argocd-initial-admin-secret -o jsonpath='{.data.password}'
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($encoded))
```

Si le mot de passe initial a déjà été changé, utiliser le mot de passe choisi. Son Secret initial peut ne plus être disponible. Ne pas réinitialiser un accès existant pour suivre la démonstration et ne conserver aucun mot de passe dans cette documentation.